Quick Start Guide: Multi-Drone Radar Simulation

Installation & Setup

Prerequisites

- **MATLAB R2023a or later**
- **Phased Array System Toolbox** (required)
- **Signal Processing Toolbox** (recommended)
- **Minimum RAM**: 8 GB (16 GB+ recommended for full 10s simulation)

File Checklist

```
multi_drone_radar_system/
├── multi_drone_radar_main.m        ← Main simulation (RUN THIS FIRST)
├── multiDroneMotion.m               ← Helper function (required)
├── beamforming_analysis.m           ← Beam analysis (optional, run after main)
├── README.md                        ← Full documentation
└── QUICKSTART.md                    ← This file
```

Installation Steps

1. Copy all `.m` files to same folder
2. Open MATLAB
3. Set working directory to simulation folder
4. Run: `multi_drone_radar_main`

Running the Simulation

Option 1: Full 10-Second Run (Default)

```
>> multi_drone_radar_main
% Outputs: 7 figures, console statistics
% Time: ~10-15 minutes
```

Option 2: Quick Test (2 seconds)

Edit `multi_drone_radar_main.m`, line 129:

```
totalDuration = 2;    % CHANGE FROM: 10
```

Then run:

```
>> multi_drone_radar_main
% Time: ~2-3 minutes (good for debugging)
```

Option 3: Custom Configuration

Use the configuration section below.

Parameter Configuration

Scenario Setup

File: `multi_drone_radar_main.m`, lines 50-80

```
%% ========== RADAR PARAMETERS ==========

fs = 25e6;                          % Sample rate [Hz]
prf = 5e3;                          % PRF [Hz] (pulses/sec)
fc = 10e9;                          % Carrier [Hz] (10 GHz)
sweepBW = 10e6;                     % Chirp bandwidth [Hz]

%% ========== RANDOM DRONE INITIALIZATION ==========

Ndrones = 4;                        % Number of drones to simulate
rng(42);                            % Random seed (for reproducibility)

% Position ranges:
% X: 300-800 m, Y: -200 to +200 m, Z: 80-150 m

% Velocity ranges:
% Vx, Vy: ±10 m/s, Vz: ±2 m/s
```

Array Configuration

File: `multi_drone_radar_main.m`, line 144

```
ura = phased.URA( ...
    'Size', [8 8], ...              % [rows cols] (8x8 = 64 elements)
    'ElementSpacing', [lambda/2 lambda/2]);
```

Simulation Duration & Resolution

File: `multi_drone_radar_main.m`, line 129

```
totalDuration = 10;                 % Simulation time [seconds]
% Automatically calculated:
% Niter = totalDuration × prf
```

Common Configurations

Config A: Quick Proof-of-Concept (2 min)

```
Ndrones = 2;
totalDuration = 1;
ura.Size = [4 4];
prf = 2e3;
sweepBW = 5e6;
```

Config B: Standard Multi-Drone Test (5 min)

```
Ndrones = 4;
totalDuration = 3;
ura.Size = [8 8];
prf = 5e3;
sweepBW = 10e6;         % ← DEFAULT (current setting)
```

Config C: High-Resolution (15 min)

```
Ndrones = 4;
totalDuration = 10;
ura.Size = [16 16];     % 256 elements!
prf = 10e3;
sweepBW = 20e6;         % Higher BW = better range res
fc = 15e9;              % Shorter wavelength = narrower beams
```

Config D: Low-Frequency Variant (5 min)

```
fc = 5e9;               % 5 GHz (S-band)
sweepBW = 5e6;
prf = 3e3;
ura.Size = [8 8];
```

Understanding the Output

Figure 1: Range Tracking

```
Y-axis: Range from radar (500-1000 m typical)
X-axis: Time (0-10 sec)

WHAT TO LOOK FOR:
✓ 4 distinct colored tracks (one per drone)
✓ Smooth curves (not jagged)
✓ Different slopes (different closing/opening rates)
✓ Overlay of cyan dots (matched filter detections)
```

Figure 2: Radial Speed

```
Y-axis: Radial velocity (m/s, ±50 typical)
X-axis: Time (0-10 sec)

WHAT TO LOOK FOR:
✓ Positive = receding, Negative = approaching
✓ Different drones have different speeds
✓ May show small variations (rotor modulation)
```

Figure 3: Micro-Doppler Spectrogram

```
X-axis: Time (seconds)
Y-axis: Doppler frequency (±100 Hz typical)
Color: Signal power (dB)

WHAT TO LOOK FOR:
✓ Horizontal "smear" around 0 Hz (body Doppler)
✓ Fine modulation lines (blade rotation sidebands)
✓ Spacing between lines ≈ 10 Hz (blade flash frequency)
```

Figure 4: SNR Evolution

```
Y-axis: Signal power (dB, typically -30 to 0)
X-axis: Time (seconds)

WHAT TO LOOK FOR:
✓ Higher SNR when drone is closer
✓ Lower SNR as drone recedes
✓ Each drone has unique SNR profile
```

Figure 5: 3D Flight Paths

```
X, Y, Z axes: Position in meters
- Black square: Radar location
- Circles: Drone start positions
- Squares: Drone end positions
- Lines: Flight paths

WHAT TO LOOK FOR:
✓ 4 distinct paths radiating from radar
✓ Smooth trajectories (no sharp kinks)
✓ Mix of approaching and receding motions
```

Figure 6: Beamforming Angles (Azimuth & Elevation)

```
Two subplots:
1. Azimuth steering angle (degrees, -90 to +90)
2. Elevation steering angle (degrees, 0 to 90)

WHAT TO LOOK FOR:
✓ Smooth changes (adaptive steering working)
✓ Rapid transitions indicate beam switching
✓ Elevation typically 10-30° (drones above radar)
```

Figure 7: Radiation Patterns

```
4 subplots showing antenna gain patterns
- Main lobe (narrow, 3° wide typical)
- Sidelobes (lower level, suppressed by array)
- Steer direction marked (angle on axis)

WHAT TO LOOK FOR:
✓ Pattern narrows and peaks toward steer angle
✓ Nulls appear away from main beam
✓ Sidelobe level ~25-30 dB below main lobe
```

Beamforming Analysis Output

After running `beamforming_analysis.m`:

Figure 1: Individual Beam Patterns (4×1)

- Shows one beam for each of the 4 drones
- Demonstrates isolation between beams

Figure 2: 3D Beam Pattern

- Full 3D gain surface
- Shows coverage and nulls

Figure 3: Multi-Beam Gain Response

- Overlaid beams across azimuth
- Shows separation capability

Figure 4: Polar Radiation Patterns

- Elevation slice in polar coordinates
- Traditional radar representation

Figure 5: Beamforming Gain Analysis

- On-axis vs off-axis comparison
- Sidelobe suppression metrics

Figure 6: Spatial Resolution

- 3dB beamwidth measurement
- Angular separation capability

Figure 7: Beam Steering Dynamics (Heatmap)

- 2D plot of gain vs steer vs scan angle
- Reveals beam shape across coverage

Figure 8: Adaptive Weights Visualization

- Left: Element weight magnitude (gradient)
- Right: Element weight phase (color)
- Shows how array elements combine

Figure 9: Multi-Beam Simultaneous Tracking

- Main beam steered to Drone 1
- Shows interference from other drones
- Demonstrates interference rejection

Interpreting Key Metrics

Range Resolution

```
Δr = c / (2 × BW)
    = 3×10⁸ m/s / (2 × 10×10⁶ Hz)
    = 15 meters
```

Meaning: Can distinguish targets 15 m apart in range

Velocity Resolution

```
Δv = λ × PRF / 2
    = 0.03 m × 5000 Hz / 2
    ≈ 75 m/s (max unambiguous velocity)
```

Meaning: Can distinguish velocities up to ±75 m/s (no folding for our drones)

Beamwidth

```
θ_3dB ≈ λ / D
       ≈ 0.03 m / 1.2 m
       ≈ 1.4° (simplified formula)
```

Actual: ~3° for 8×8 array (accounting for element spacing)
Meaning: Can separate targets ~3° apart in angle

Array Gain

```
G_array = 10 × log₁₀(N_elements)
        = 10 × log₁₀(64)
        = 18.06 dB
```

Meaning: Coherent combination improves SNR by factor of 64

Detection Range

```
R_max ≈ ⁴√(P_tx × G_tx × G_rx × G_array × RCS / P_noise)
      ≈ 1000+ meters (for drones with RCS 0.2 m²)
```

Meaning: Can detect drones well beyond our simulation volume

Troubleshooting Guide

Problem: "Out of Memory" Error

```
Error: Requested array exceeds maximum array size limit.
```

Solution:

- Reduce `totalDuration` (line 129): `totalDuration = 2;`
- Reduce `fs` (line 51): `fs = 12.5e6;` (half sample rate)
- Reduce array size (line 144): `ura.Size = [4 4];`

Problem: Simulation Runs Very Slowly

```
Elapsed time: 45 seconds for 1000 pulses...
```

Solution:

- Run on faster computer (GPU acceleration not yet implemented)
- Reduce simulation duration
- Disable plotting during simulation (comment out plotting section)

Problem: "Undefined Function" Error

```
Error: Undefined function or variable 'multiDroneMotion'
```

Solution:

- Ensure `multiDroneMotion.m` is in same folder as main script
- Check file is named exactly as shown (case-sensitive on Linux)
- Type: `which multiDroneMotion` to verify MATLAB can find it

Problem: Plots Look Wrong (All Zeros or Flat)

```
Range tracking shows constant 0 meters, or all plots are flat.
```

Solution:

- Check PRF setting (line 52): should be reasonable (1k-10k Hz)
- Verify drone positions initialized correctly (line 67)
- Run with `Ndrones = 1` first (simpler case)
- Check console for warnings during simulation

Problem: "Beamforming Analysis" Won't Run

```
Error: Variables 'rxsig' or 'ura' not found
```

Solution:

- First run: `multi_drone_radar_main`
- This populates required variables

- Then run: `beamforming_analysis`
- Or run `beamforming_analysis` standalone (it creates its own `ura`)

Performance Tips

For Faster Simulation

1. **Reduce sample rate**: `fs = 12.5e6` (from 25e6)
2. **Reduce PRF**: `prf = 2e3` (from 5e3)
3. **Reduce array size**: `[4 4]` (from 8×8)
4. **Reduce chirp bandwidth**: `sweepBW = 5e6` (from 10e6)

For Better Accuracy

1. **Increase sample rate**: `fs = 50e6` (from 25e6)
2. **Increase PRF**: `prf = 10e3` (from 5e3)
3. **Increase array size**: `[16 16]` (from 8×8)
4. **Longer simulation**: `totalDuration = 30`

Memory Usage Estimate

```
MEMORY = Nsamples_per_pulse × Niterations × bytes_per_sample
       = (fs/prf) × (duration × prf) × 8
       = fs × duration × 8

Examples:
- 25 MHz, 10 sec: 2 GB
- 25 MHz,  3 sec: 0.6 GB
- 12.5 MHz, 10 sec: 1 GB
```

Next Steps

After First Successful Run:

1. ✅ Verify plots match expected results
2. ✅ Run `beamforming_analysis` for detailed beam study
3. ✅ Experiment with parameter variations (Config A, B, C, D)
4. ✅ Read detailed explanation in README.md

Research Directions:

- **Adaptive Beamforming**: Implement MVDR (Capon's method) in code
- **Clutter Suppression**: Add MTI/STAP processing
- **Target Tracking**: Add Kalman filter for angle prediction
- **Classification**: Use micro-Doppler signature to identify drone type
- **Phased Array Optimization**: Investigate 2D FFT for simultaneous multi-beam

---

MATLAB Cheat Sheet

Useful Commands During Simulation

```
% Pause simulation at any time
CTRL+C

% Clear all variables
clear all

% List current workspace
whos

% Check array element count
>> Nelements
Nelements = 64

% Verify PRF setting
>> prf
prf = 5000

% Check range resolution
>> c/(2*sweepBW)
ans = 15

% Plot stored position history
>> figure; plot(squeeze(dronePos(1,:,:))'); xlabel('Time'); ylabel('X (m)');

% Export figure as high-res image
>> saveas(gcf, 'my_figure.png')

% Export workspace to file
>> save my_simulation_results.mat

% Reload saved variables
>> load my_simulation_results.mat
```

Contact & Support

For issues or questions:

1. Check inline comments in `.m` files
2. Review README.md detailed explanation
3. Check MATLAB help: `help phased.URA`
4. Enable debugging: Add `fprintf()` statements in main loop

Happy Simulating! 🎯 📡 ✈️

Last Updated: 2026